أكاديمية الفضاء الوطنية
NATIONAL SPACE ACADEMY

وكالة الإمارات للفضاء
UAE SPACE AGENCY



GDAL End to End

Muhammed Sirajul Huda K · Space42 · Space Academy 2026

Why GDAL? One library instead of thousands of readers

why it exists

Formats

- GeoTIFF
- COG (cloud)
- JPEG 2000
- NetCDF · HDF5
- Sentinel-2 SAFE
- GeoPackage
- Shapefile
- GeoJSON

GDAL / OGR

reads and writes the formats

PROJ

handles the coordinates

Tools you know

- python
- QGIS
- ArcGIS
- GeoPandas
- PostGIS
- Tiler
- TorchGeo

Without GDAL

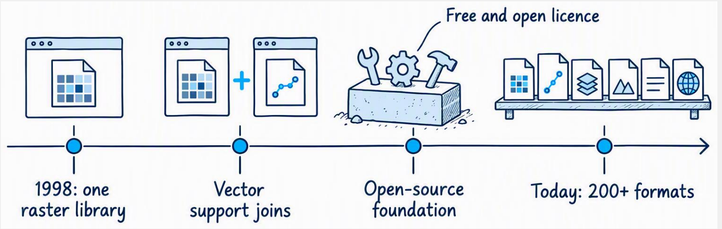
Every tool writes its own reader for every format.
10 tools × 200 formats = 2,000 readers to build and fix

With GDAL

Each reader is written once, inside GDAL.
10 tools + 200 formats: every tool gets every format

Illustrative numbers

- **Free and open since 1998:** an OSGeo project with an MIT-style licence
- **Paid for by the people who rely on it:** sponsors include Esri, Planet and Google
- **One data model for raster and vector,** one coordinate engine (PROJ)
- **Fix a bug once in GDAL,** and every tool above it is fixed



This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Why Python needs GDAL

Your code

jupyter python™

Python libraries

rasterio	xarray	GeoPandas	TorchGeo
Rasterio	**rioxarray**	**GeoPandas**	**TorchGeo**
→ GDAL	→ Rasterio → GDAL	→ pyogrio → OGR	→ Rasterio → GDAL

GDAL (C/C++)

GDAL PROJ

decodes 200+ formats
coordinates and reprojection
windows read from the cloud

Your data

GeoTIFF | COG in the cloud | GeoPackage | NetCDF

Rasterio · GeoPandas · xarray

NumPy **knows arrays, not maps**

- **No geography in an array:** no coordinate system, no geotransform, no file format
- **Decoding is C/C++ work:** GeoTIFF, JPEG 2000, NetCDF, compression and cloud reads, built into GDAL since 1998
- **pip install rasterio** downloads a wheel that carries its own copy of GDAL
- **Same library, same answer:** rasterio.open() and gdal.Open() read through GDAL, traps included

Same file, same library, two doors

```
                                # Rasterio: friendly Python
import rasterio
with rasterio.open("B04.tif") as src:
    red = src.read(1)                # NumPy array
    print(src.crs, src.transform)

# GDAL Python API: closer to the engine
from osgeo import gdal
ds = gdal.Open("B04.tif")
red = ds.GetRasterBand(1).ReadAsArray()
print(ds.GetProjection(), ds.GetGeoTransform())
```

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Why QGIS, ArcGIS and map servers need GDAL

desktop · database · web

QGIS

Desktop GIS · free

Every raster and vector layer opens through its GDAL and OGR data providers.
Processing › GDAL gives gdalwarp and gdal_translate a form.

Source: QGIS code, src/core/providers/gdal and /ogr

ArcGIS Pro

Desktop GIS · Esri

Esri: “Most raster formats are supported using the Geospatial Data Abstraction Library (GDAL).”
Esri is one of GDAL's sponsors.

Source: ArcGIS Pro help, supported raster formats · gdal.org sponsors

ArcMap

ArcGIS

Desktop GIS · Esri, older

Esri: deprecated and in mature support; 10.8.x is the final release series. Move projects to ArcGIS Pro; GeoTIFF and shapefile outputs open in both.

Source: Esri ArcGIS Desktop life-cycle page

GRASS GIS

Analysis engine · free

Imports and exports through GDAL and OGR:
r.in.gdal, r.out.gdal, v.in.ogr, v.out.ogr.

Source: GRASS GIS module manual

PostGIS

PostGIS
Spatial PostgreSQL

Spatial database

raster2pgsql loads any raster GDAL can read.
ST_FromGDALRaster and ST_AsGDALRaster convert formats inside SQL.

Source: PostGIS raster reference

TiTiler

Web map tile server

Serves map tiles straight from COGs:
TiTiler → rio-tiler → Rasterio → GDAL, with the cloud reads of Example 11.

Source: TiTiler and rio-tiler documentation

Learn the GDAL command once, and you know what the buttons in all of these tools are doing.

Logos are trademarks of their owners, shown to identify the software.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Why GDAL matters for agentic AI

agents need tools

Question

“How green is Al Ain this month?”

AI agent

plans · calls tools · answers
cloud or local model

Answer + evidence

mean NDVI, which scene,
which date, how cloudy

Mission 4 notebook: a geo agent built exactly this way

Model Context Protocol

→ tool call: name + JSON arguments
← result: a few KB of JSON

GDAL

```
gdal raster info
  --format json
/vsicurl/ window read
gdal_calc.py NDVI
gdalwarp -cutline
```

GDAL tools

Sentinel-2 COGs in the cloud · 241 MB per band

- **Pixels don't fit in a prompt.** One 10 m band is 241 MB. GDAL reads the pixels; the model reads a summary of a few KB.
- **Structured in, structured out.** gdal raster info --format json returns facts a model can parse; --json-usage describes every command.
- **Deterministic and auditable.** The same command gives the same result, so every step can be logged and re-run by a person.
- **Any model, cloud or local.** Wrap the commands once as tools, for example with MCP, then swap the model freely.
- **Guardrails live in the tool.** Check CRS, pixel size and cloud cover in code before the model answers.
- **Test what the model writes.** A generated command can still hit the integer NDVI trap from Example 6.

```
                    $ gdal raster reproject --json-usage       # GDAL 3.11+
{ "name": "reproject",
  "description": "Reproject a raster dataset.",
  "input_arguments": [ … ] }                # a ready-made tool description
```

```
                          $ gdal raster info --format json water_B04.tif
{ "size": [848, 700],
  "geoTransform": [54.574585, 8.99e-05, 0, 24.598628, 0, -8.17e-05],
  "bands": [{ "type": "Byte" }] … }         # what the model reads: a few KB
```

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

GDAL and the tools you already know

comparison

Tool	What it is	Uses GDAL?	Best for
GDAL / OGR command line	the library itself, ~50 small programs	it is GDAL	pipelines, servers, batch jobs
GDAL Python API (osgeo)	direct bindings to the same C++ code	yes	exact control, debugging
Rasterio · Fiona · pyogrio	Pythonic wrappers, NumPy in and out	yes	analysis code that stays readable
GeoPandas · xarray / rioxarray	data frames and labelled arrays	yes, underneath	tables and time series
QGIS	free desktop GIS	yes, for every layer	looking, digitising, quick maps
ArcGIS Pro	commercial desktop GIS	yes, for many formats	enterprise workflows, cartography
PostGIS (raster)	spatial database	yes	many users, one source of truth
Google Earth Engine	Google's cloud platform	no: its own engine	global scale, no local storage

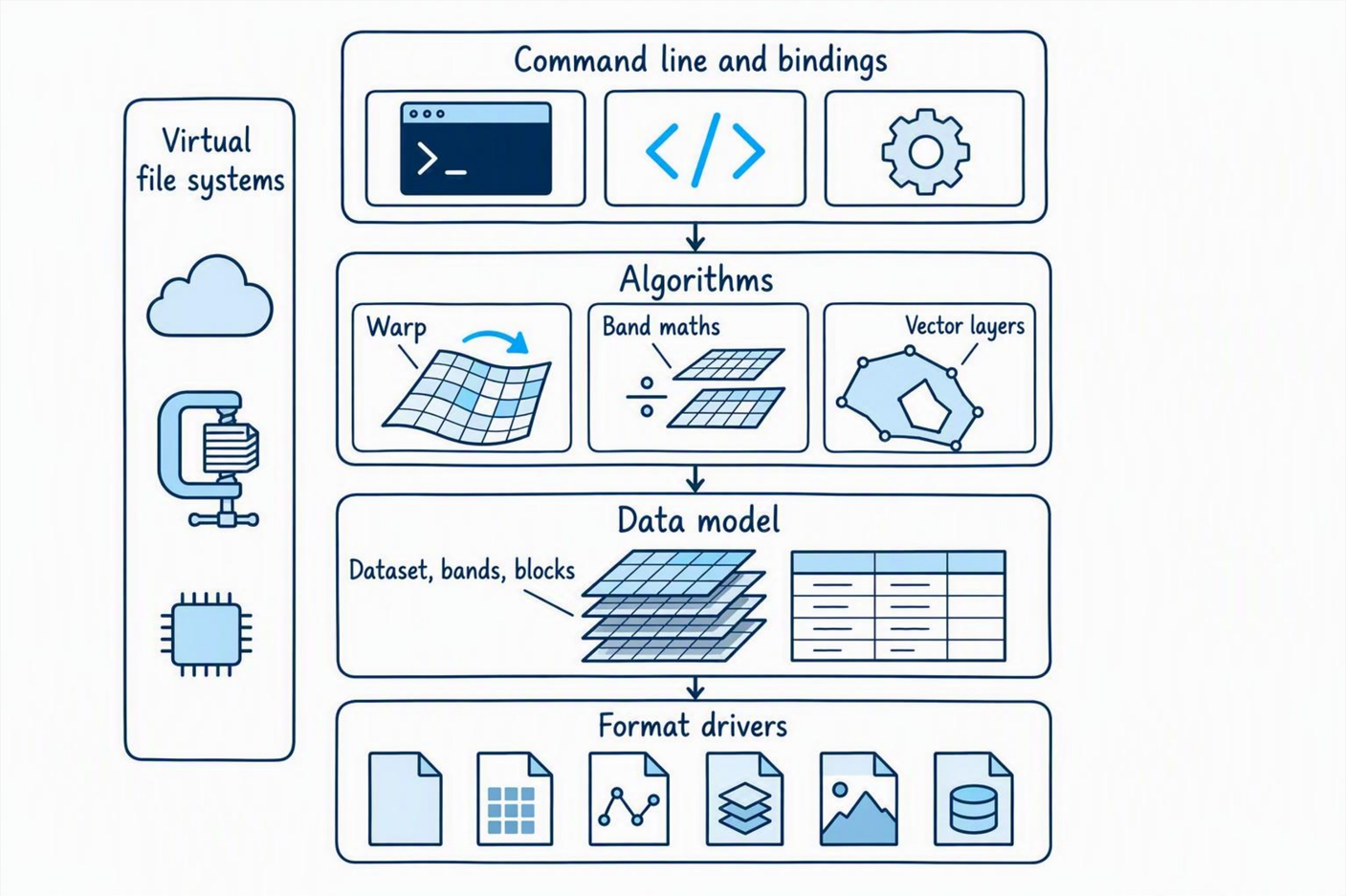
- Rule of thumb: click in QGIS to explore, write GDAL commands when it must run again tomorrow

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

How GDAL is built

architecture

- Format drivers: GTiff, JP2, NetCDF, HDF5, GPKG, Shapefile … one per format
- Data model: Dataset → bands → blocks, plus geotransform, CRS, metadata
- Algorithms work on the model: warp, band maths, rasterize, polygonize, contour
- Bindings and ~50 command-line programs sit on top: same engine, different door
- Virtual file systems make a URL, a zip or memory look like a normal file path

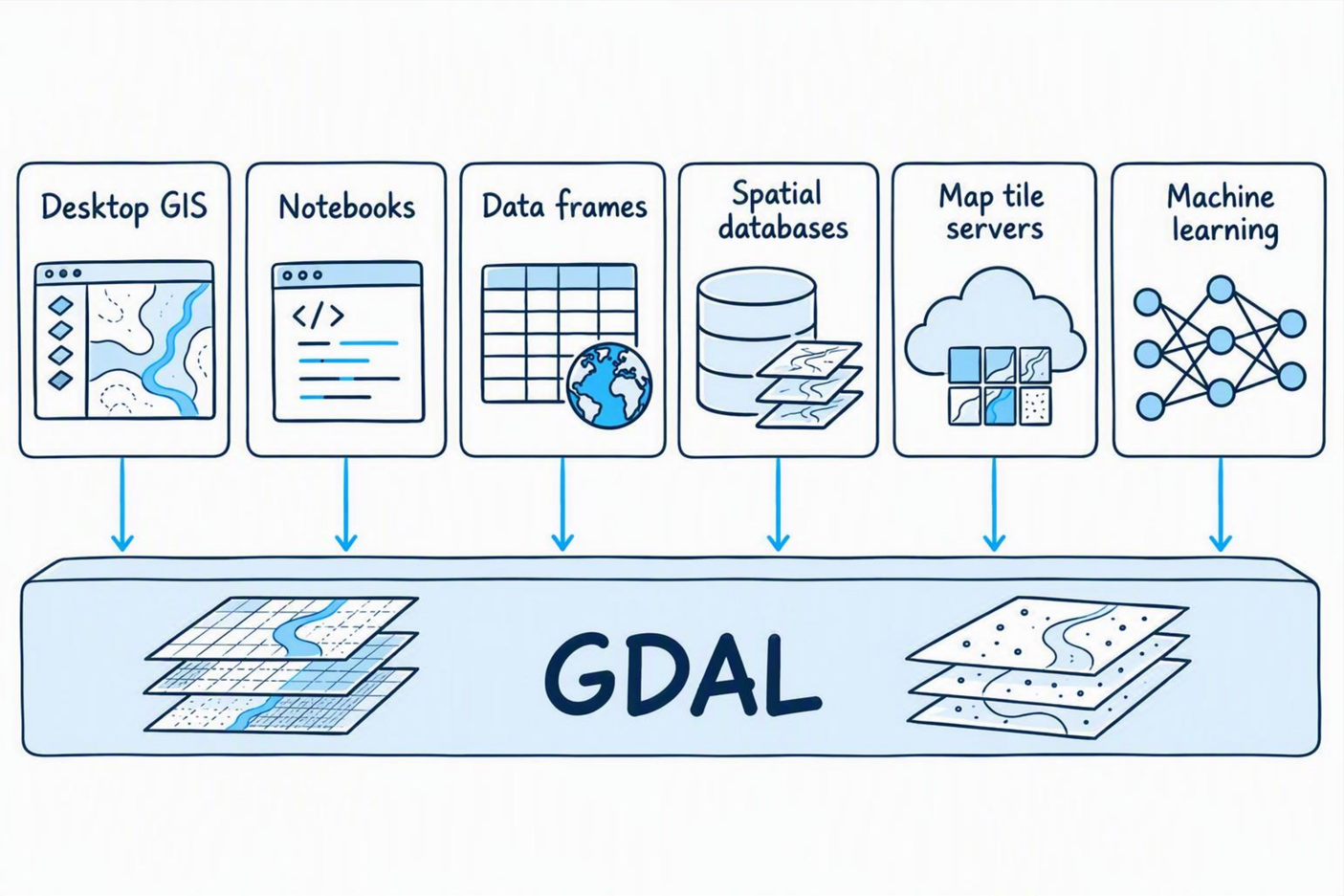


This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

How modern tools use GDAL

the stack today

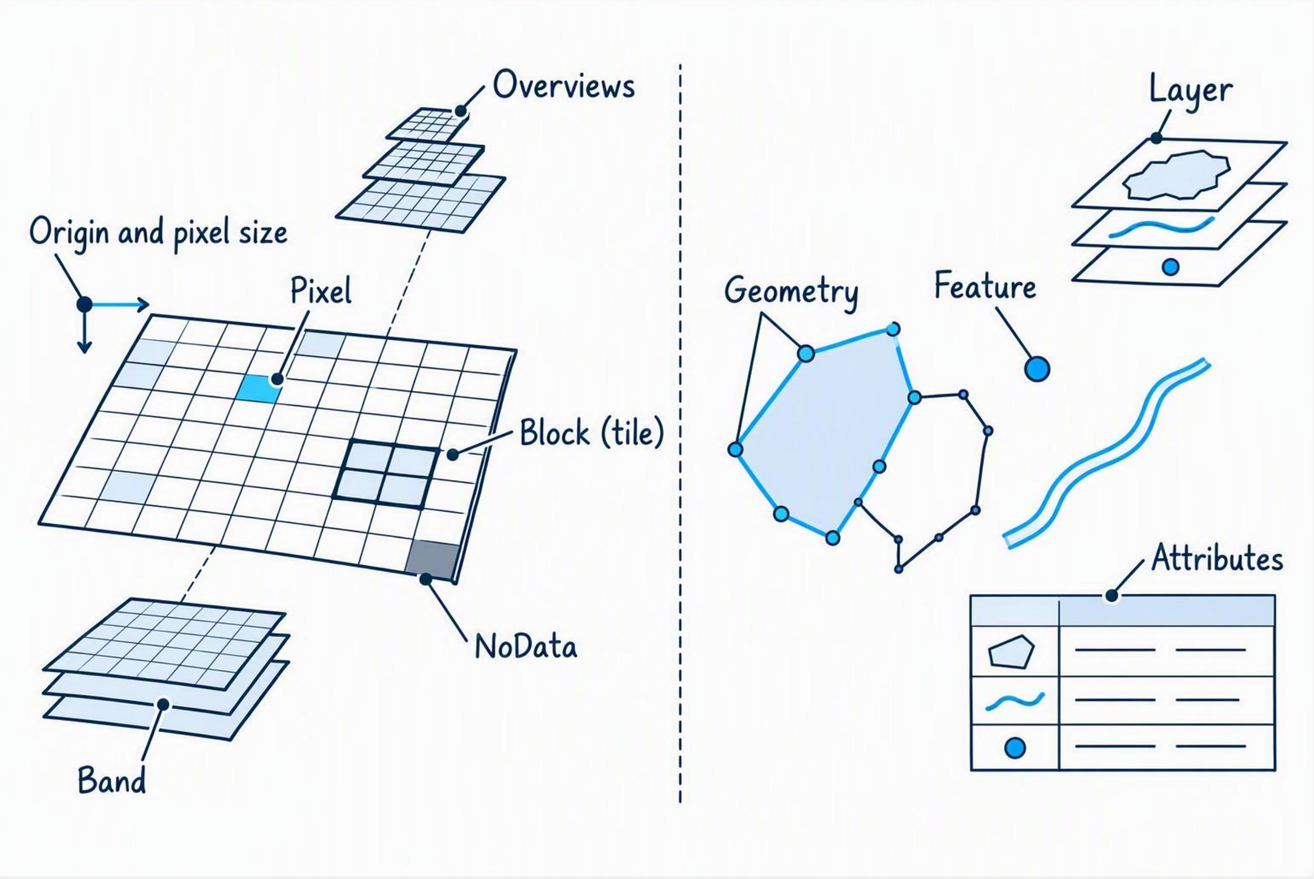
- Desktop GIS: QGIS and ArcGIS open every layer through GDAL
- Notebooks: Rasterio, rioxarray, GeoPandas wrap the same library
- Databases: PostGIS loads rasters and vectors with GDAL utilities
- Cloud: STAC catalogues + COGs + tile servers read with /vsicurl/
- Machine learning: GDAL cuts the chips and writes the predicted maps



This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

The words you need

terminology

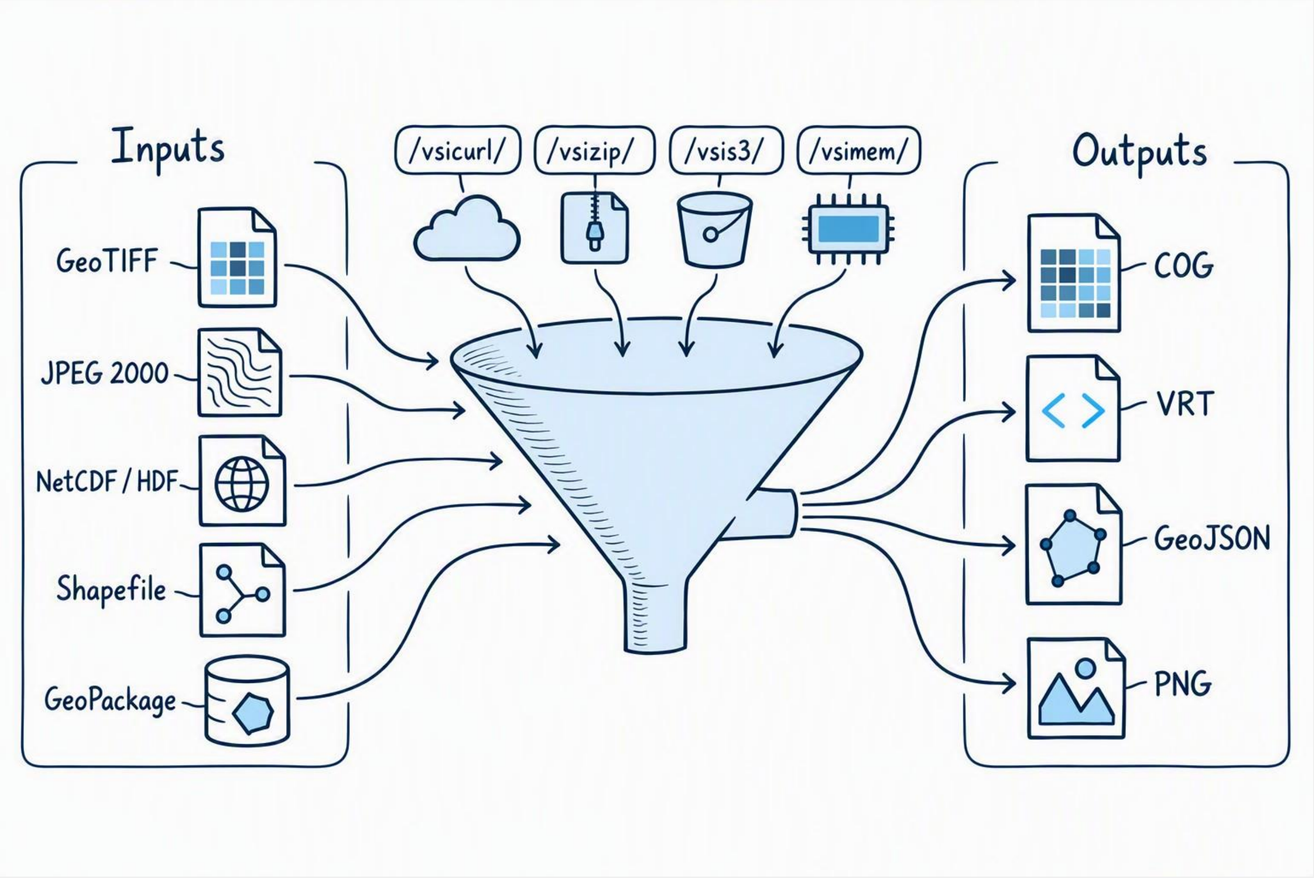


Word	Meaning
Dataset	one file as GDAL sees it: bands + geotransform + CRS
Band	one layer of values, e.g. red or NIR
Geotransform	6 numbers: origin, pixel width, pixel height
CRS / EPSG code	the coordinate system, e.g. 4326 or 32640
NoData	the value meaning 'no measurement here'
Block / tile	the chunk GDAL reads at once, e.g. 256 × 256
Overview	a smaller copy inside the file, for fast preview
Resampling	how new values are made: near, bilinear, average, cubic
VRT	a small XML recipe; no pixels copied
COG	Cloud-Optimised GeoTIFF: tiled + overviews
Driver	the code that reads or writes one format
Feature / layer	vector side: one geometry + attributes, grouped in a layer

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Input and output

`formats · virtual paths`



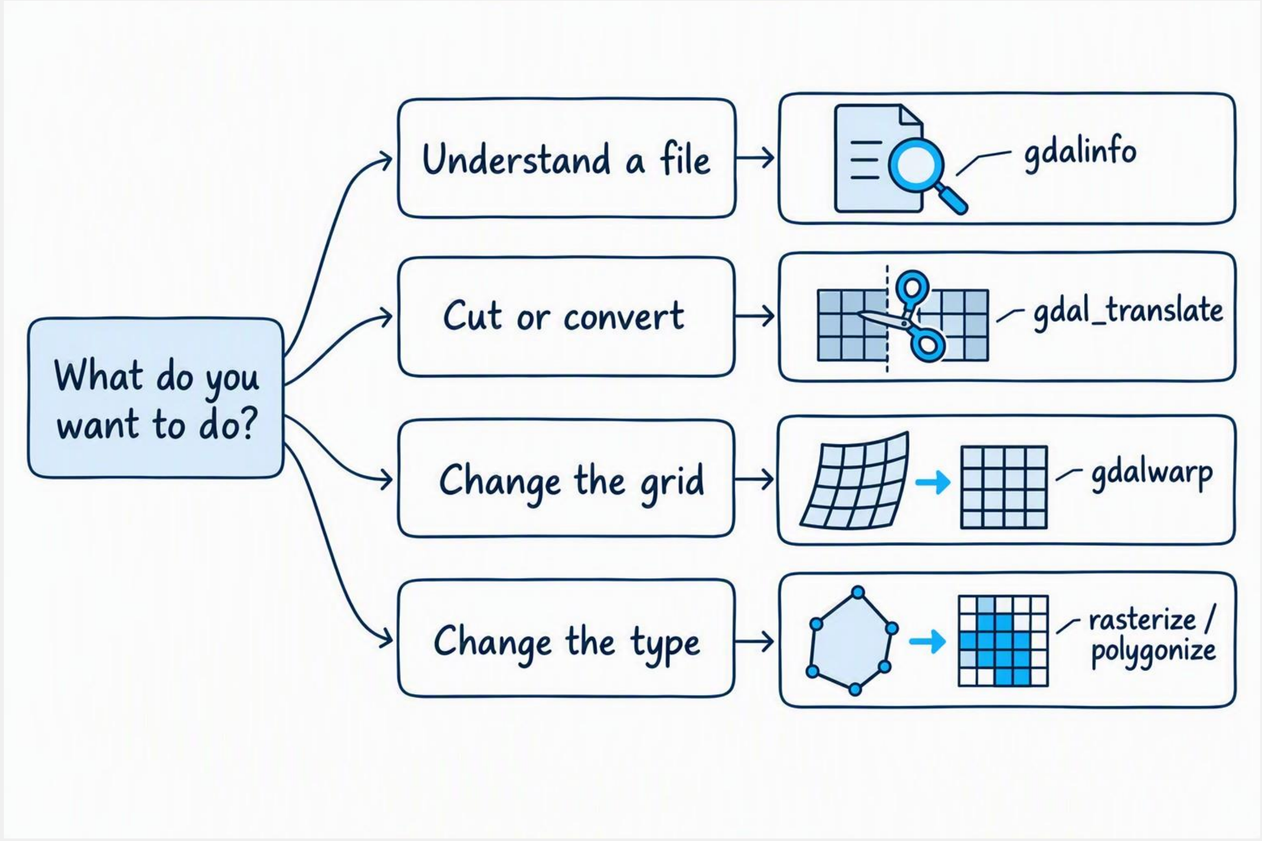
	Use	Why
GeoTIFF	working files	universal, lossless, fast
COG	anything you share	tiles + overviews, streams over HTTP
VRT	intermediate steps	a recipe, not a copy: instant
GeoPackage	vectors, new work	one file, no name limits, SQL inside
GeoJSON	small vectors, web	plain text, everyone reads it
Shapefile	only when asked	old limits: 2 GB, short field names
PNG / JPEG	slides and reports	picture only, no measurements
NetCDF / HDF	science time series	many dimensions in one file

- Virtual paths: /vsicurl/ (web) · /vsizip/ (inside a zip) · /vsis3/ (buckets) · /vsimem/ (memory)

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Which command for which job

the map



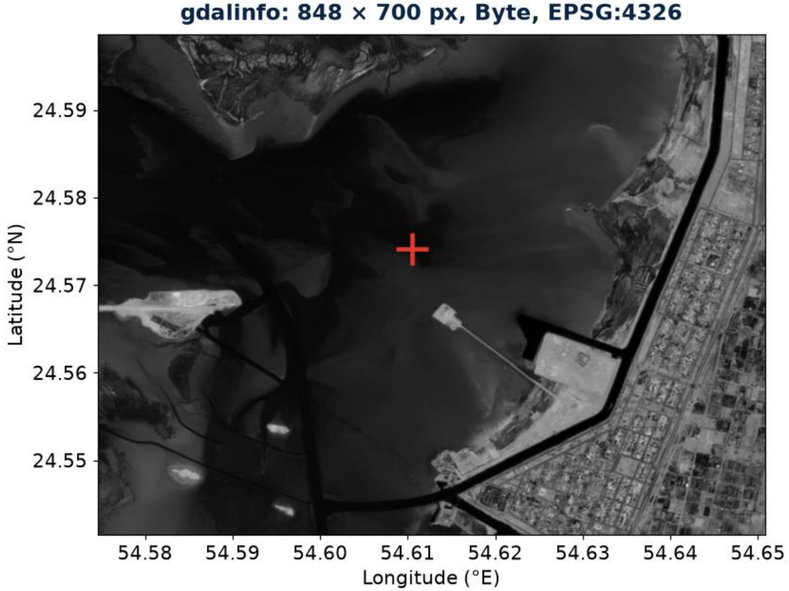
I want to…	Command
see size, CRS, bands, statistics	gdalinfo -stats -json
cut a window, convert, pick bands	gdal_translate
reproject, resample, clip, mosaic	gdalwarp
stack bands or mosaic for free	gdalbuildvrt -separate
compute an index, classify, threshold	gdal_calc.py
turn polygons into pixels	gdal_rasterize
turn pixels into polygons	gdal_polygonize.py
remove small patches from a mask	gdal_sieve.py
draw contour lines from a surface	gdal_contour
add overviews / cut chips	gdaladdo · gdal_retile.py
fix metadata without rewriting pixels	gdal_edit.py
read a file on the web	/vsicurl/ + COG

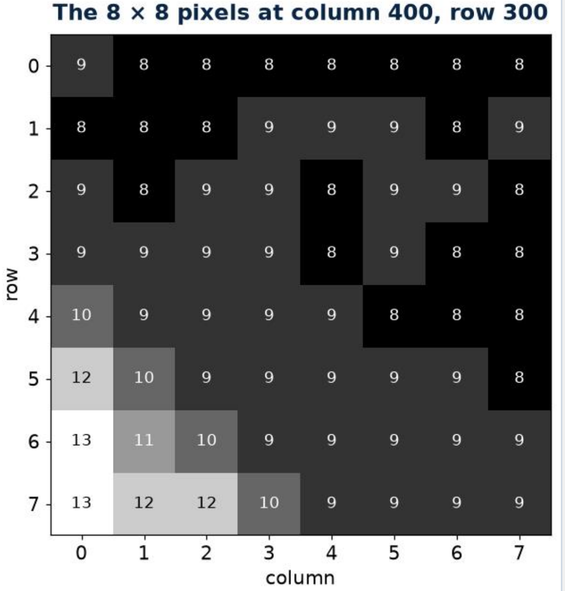
This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Example 1. Inspect before you touch

gdalinfo

- This band: 848 × 700 pixels, Byte, EPSG:4326
- The geotransform is 6 numbers: origin, pixel width, pixel height
- x = GT0 + col × GT1 → pixel (400, 300) is at 54.61054°E, 24.57413°N
- Pixel height is negative: rows go down, north stays up
- Here one pixel is about 9.1 m on the ground





IN **water_B04.tif · one Sentinel-2 red band**

OUT **metadata only (no new file) · mean 44.8, max 224**

```
gdalinfo -stats water_B04.tif
gdalinfo -json water_B04.tif

# GT = [54.5746, 0.0000899, 0,
#       24.5986, 0, -0.0000817]
```

Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Cut a piece, change the format

`gdal_translate`

Example 2 · Cut a window, by pixels or by coordinates



- -srcwin col row width height cuts by pixel position
- -projwin ulx uly lrx lry cuts by map coordinates
- 339 KB → 74 KB: every later step gets faster

IN water_B04.tif (848 × 700)

OUT 300 × 250 px · 335 × 368 px

```
gdal_translate \
 -srcwin 300 200 300 250 \
 water_B04.tif sub.tif

gdal_translate \
 -projwin 54.60 24.59 \
          54.63 24.56 \
 water_B04.tif sub2.tif
```

Example 3 · Change format and pick bands



- -of writes another format; PNG needs -scale because it is 8-bit
- -b picks and reorders bands: same pixels, different story
- NIR, red, green makes healthy fields glow red

IN water_B04.tif · agriculture_stack.tif (4 bands)

OUT PNG 257 KB · two 3-band composites

```
gdal_translate -of PNG \
 -scale in.tif out.png

gdal_translate \
 -b 4 -b 1 -b 2 \
 stack.tif cir.tif
```

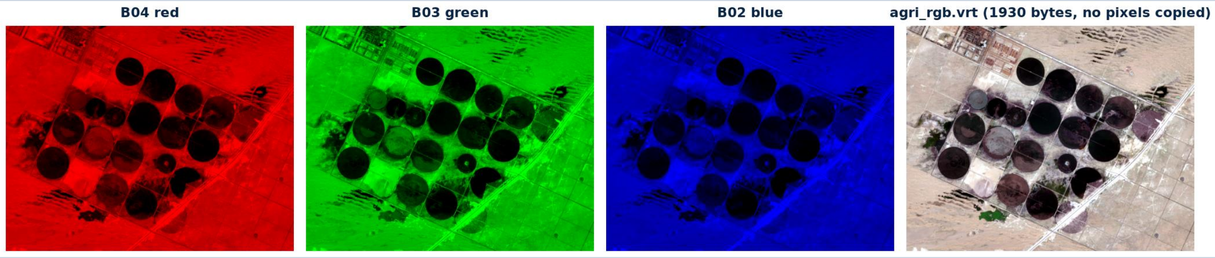
Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Stack bands and make them visible

`gdalbuildvrt · -scale`

Example 4 · Stack bands without copying pixels



- -separate stacks bands; without it GDAL puts files side by side
- The VRT is 1,930 bytes of XML; the real copy is 2,311 KB
- A VRT is a recipe: keep the band files where they are

IN B04, B03, B02 (three single-band files)

OUT agri_rgb.vrt · 1,930 bytes, behaves like a 3-band image

```
gdalbuildvrt -separate \
 rgb.vrt \
 B04.tif B03.tif B02.tif

# copies every pixel:
gdal_merge.py -separate \
 -o rgb.tif B04.tif …
```

Example 5 · Stretch for display (never for measuring)



- Raw values sit between 9 and 127: the composite looks dark
- -scale spreads them to 0–255; -exponent applies gamma
- Different band triplets answer different questions

IN agri_rgb.vrt · urban SWIR bands

OUT 8-bit pictures for slides; the raw file is untouched

```
gdal_translate -ot Byte \
 -scale_1 9 127 0 255 \
 -scale_2 15 108 0 255 \
 -scale_3 2 87 0 255 \
 rgb.vrt rgb8.tif
```

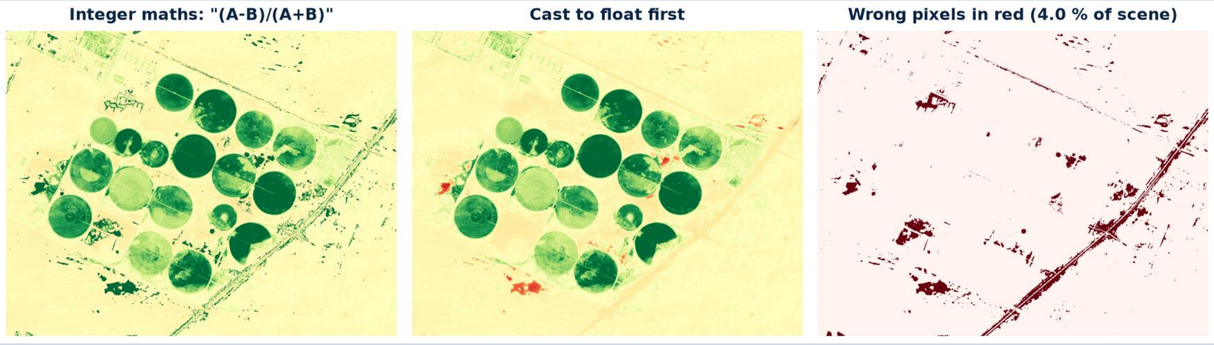
Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Band maths: from bands to a measurement

gdal_calc.py

Example 6 · NDVI, and the integer trap



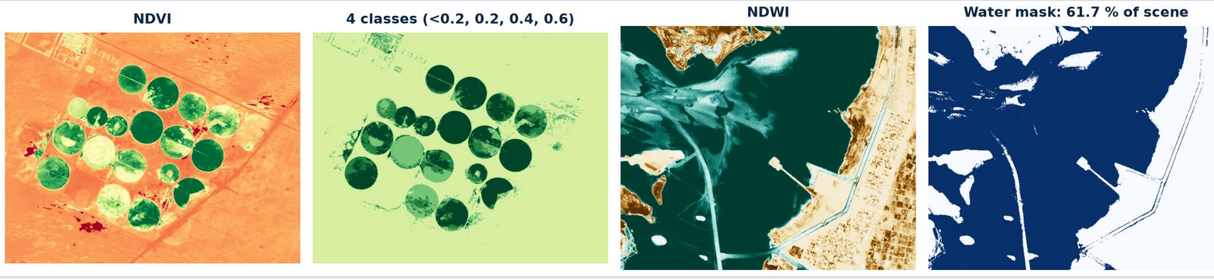
- NDVI = (NIR − Red) / (NIR + Red): one formula, any band pair
- On 8-bit integers, 4.0 % of pixels come out wrong and the maximum becomes infinity
- Cast to float and the range is right: -0.62 to 1.0

```
IN    B08 (NIR) and B04 (red), 8-bit

OUT  ndvi.tif · Float32, −1 to 1
```

```
gdal_calc.py -A B08.tif \
-B B04.tif \
--calc="(A.astype(float)-B)\
/(A.astype(float)+B)" \
--type=Float32 \
--outfile=ndvi.tif
```

Example 7 · Classify and threshold



- Multiply conditions to build classes from one index
- Here: 983 ha of moderate and dense vegetation
- A mask is just a threshold: water covers 61.7 % of the coastal scene

```
IN    ndvi.tif · ndwi.tif

OUT  ndvi_class.tif (4 classes) · water_mask.tif
(0/1)
```

```
gdal_calc.py -A ndvi.tif \
--calc="1*(A<0.2)+2*((A>=0.2)\
*(A<0.4))+3*…" \
--type=Byte

--calc="A>0.2"  # mask
```

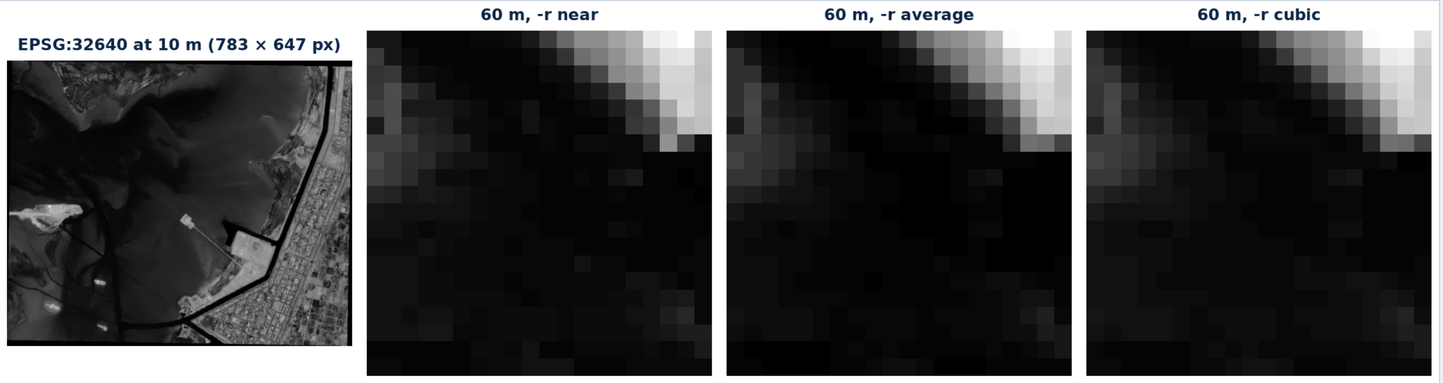
Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Example 8. Reproject and resample

`gdalwarp -t_srs -tr -r`

- Degrees → UTM metres: 848 × 700 px becomes 783 × 647 px at exactly 10 m
- -t_srs sets the target CRS, -tr the pixel size in target units
- -r near keeps class values; average and cubic smooth measurements
- Every warp resamples: do it once, as late as you can
- Measuring area or distance? Move to metres first



IN **water_B04.tif · EPSG:4326 (degrees)**

OUT **water_utm.tif · EPSG:32640, 10 m pixels (783 × 647)**

```
gdalwarp -t_srs EPSG:32640 \
  -tr 10 10 -r near \
  water_B04.tif water_utm.tif

gdalwarp -tr 60 60 -r average \
  water_utm.tif coarse.tif
```

Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Clip to a boundary, and join tiles

`gdalwarp · gdalbuildvrt`

Example 9 · Clip with a polygon



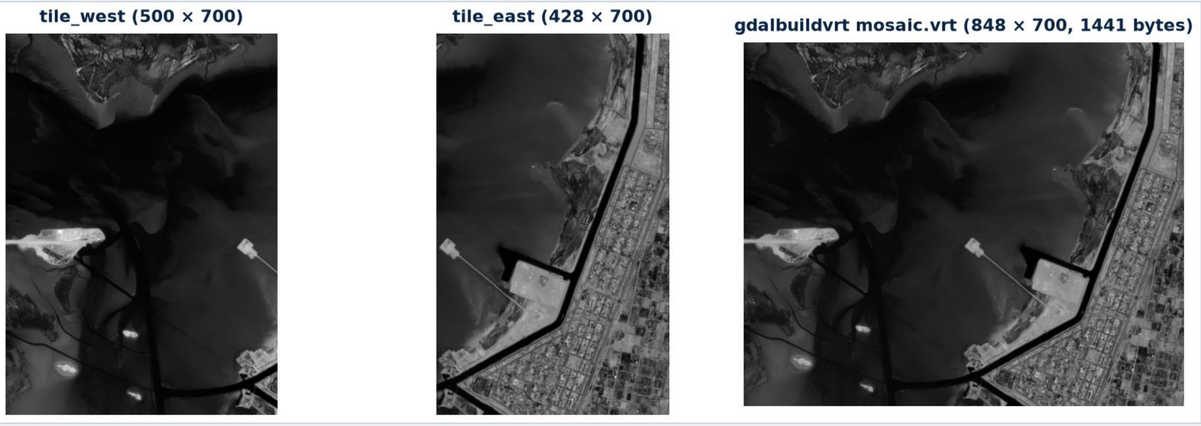
- -cutline takes any vector file and reprojects it for you
- -crop_to_cutline shrinks the extent; without it you keep the full frame
- Always set -dstnodata, or the outside becomes 0 and pollutes statistics

IN water_utm.tif (783 × 647) + aoi.geojson

OUT water_clip.tif (322 × 351), outside = NoData

```
gdalwarp \
 -cutline aoi.geojson \
 -crop_to_cutline \
 -dstnodata 0 \
 in.tif clip.tif
```

Example 10 · Mosaic tiles onto one grid



- gdalbuildvrt joins tiles instantly, if they share a CRS
- In the overlap, the last file listed wins: order by quality
- Across CRS or pixel sizes, only gdalwarp works

IN two overlapping tiles (500 × 700, 428 × 700)

OUT mosaic.vrt (848 × 700) · 1,441 bytes

```
gdalbuildvrt mosaic.vrt \
 west.tif east.tif

gdalwarp \
 -t_srs EPSG:32640 \
 -tr 10 10 \
 west.tif east.tif \
 mosaic.tif
```

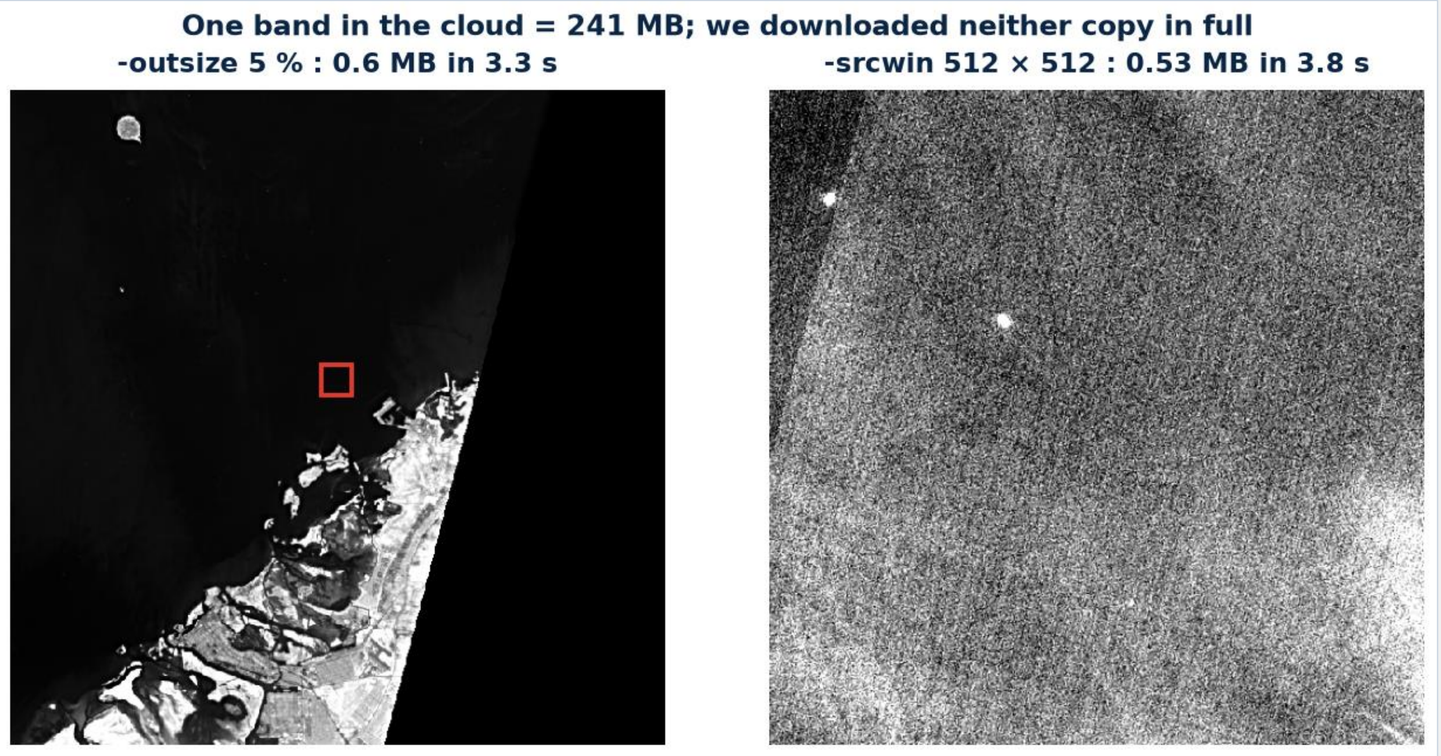
Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Example 11. Read a cloud file without downloading it

/vsicurl/ · COG

- This band in the cloud: 10,980 × 10,980 px ≈ 241 MB
- A 512 × 512 window: 0.53 MB in 3.8 s
- A 5 % preview: 0.6 MB in 3.3 s, read from the 4 overviews inside the file
- /vsicurl/ puts a URL where a filename goes
- It works because the file is a COG: tiles + overviews



IN a Sentinel-2 band in a public bucket (S2B_40RBN_20260316_0_L2A)
OUT two small local files; the 241 MB original was never downloaded

```
gdalinfo /vsicurl/https://…/B04.tif

gdal_translate \
  -srcwin 5200 4600 512 512 \
  /vsicurl/https://…/B04.tif win.tif

gdal_translate -outsize 5% 5% \
  /vsicurl/https://…/B04.tif prev.tif
```

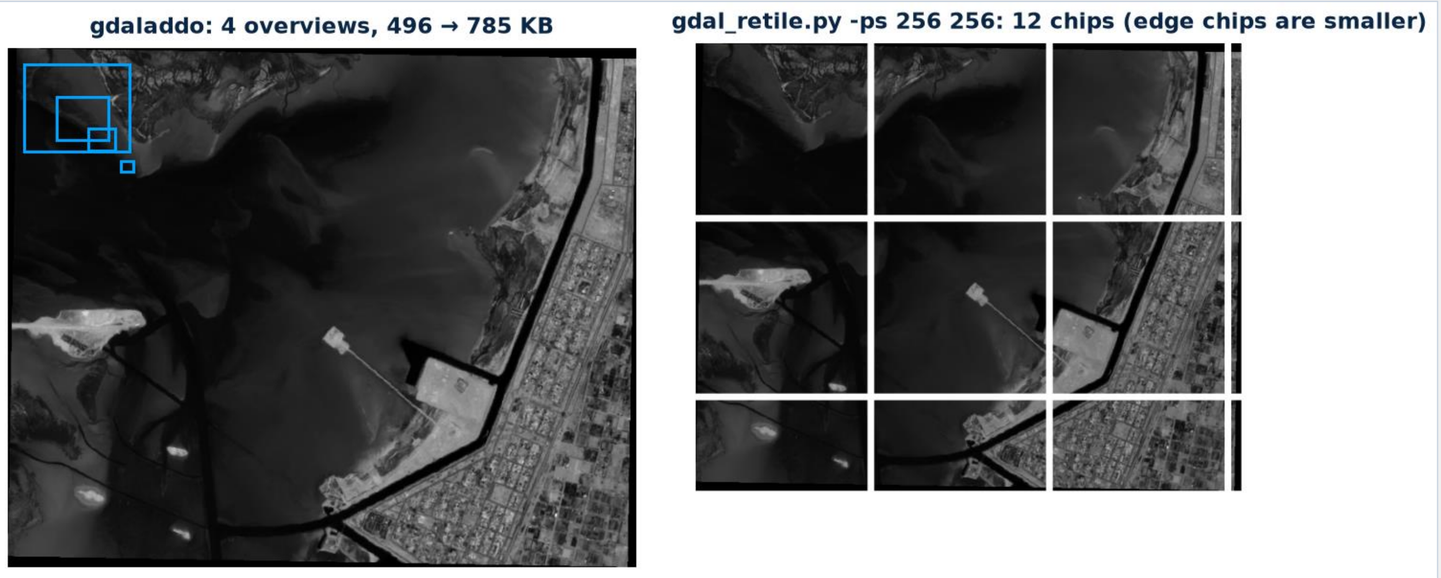
Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Example 12. Overviews and training chips

gdaladdo · gdal_retile.py

- gdaladdo added 4 smaller copies inside the file: 496 → 785 KB
- That is the trade: a little more disk for instant previews
- gdal_retile.py cut the scene into 12 chips of 256 × 256 px
- Chips are the usual first step for deep learning
- Add -overlap so objects on a border also appear whole in a neighbour



IN **water_utm.tif (496 KB)**

OUT **same file + 4 overviews (785 KB) · 12 chips**

```
gdaladdo -r average water.tif 2 4 8 16

gdal_retile.py -ps 256 256 \
  -targetDir chips/ water.tif

# with overlap for training:
gdal_retile.py -ps 256 256 \
  -overlap 32 -targetDir chips/ water.tif
```

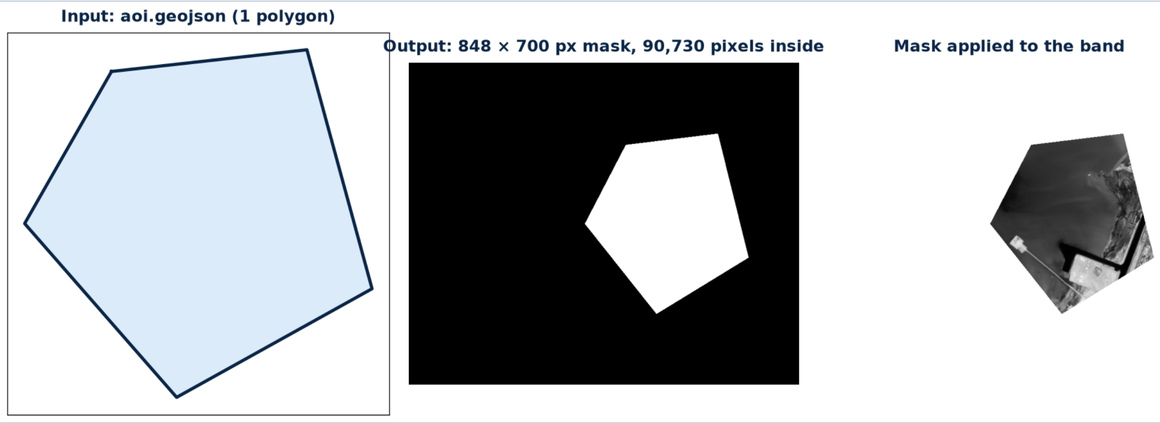
Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Between raster and vector

rasterize · sieve · polygonize · contour

Example 13 · Vector → raster



- -burn writes a fixed value; -a field writes an attribute
- Match the grid with -ts (size) and -te (extent) or the layers will not line up
- Here: 90,730 pixels inside the polygon ≈ 7.5 km²

```
gdal_rasterize \
-burn 1 -ot Byte \
-ts 848 700 \
-te xmin ymin xmax ymax \
aoi.geojson mask.tif
```

IN aoi.geojson (1 polygon) + a target grid

OUT aoi_mask.tif · 848 × 700 px, 1 inside / 0 outside

Example 14 · Raster → vector, cleaned



- gdal_sieve removed the noise: 419 patches → 14
- Sieve first, then polygonize: otherwise you make thousands of tiny polygons
- gdal_contour drew 31 shoreline lines from the NDWI surface

```
gdal_sieve.py -st 50 -4 \
 mask.tif clean.tif

gdal_polygonize.py clean.tif \
 -f GPKG water.gpkg

gdal_contour -a ndwi \
 -fl 0.2 ndwi.tif shore.gpkg
```

IN water_mask.tif (0/1) · ndwi.tif (continuous)

OUT GeoPackages: water polygons and a shoreline you can open in QGIS

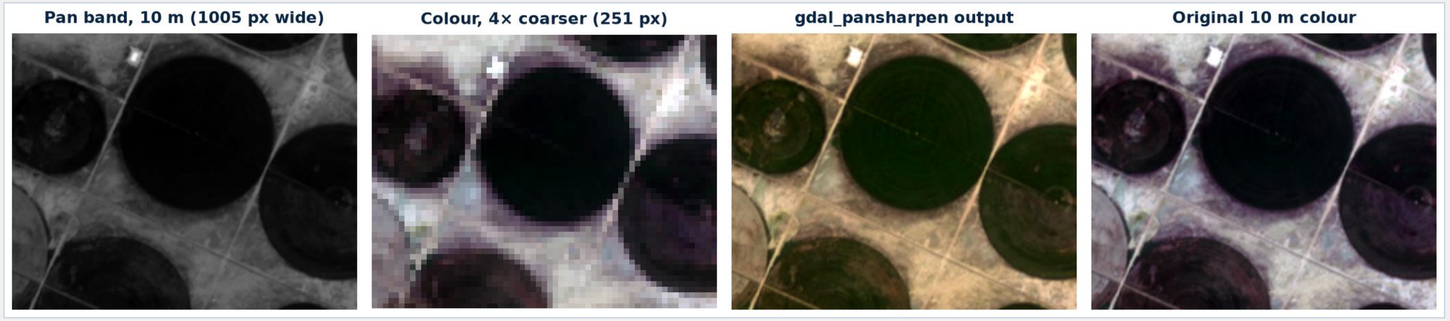
Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Example 15. Pan-sharpening

gdal_pansharpen.py

- Some satellites carry one sharp grey band plus coarser colour bands
- Pan-sharpening puts the detail of the grey band into the colour bands
- Sentinel-2 has no pan band, so we simulated one: colour at 251 px became 1005 px
- The result is sharper, and the colours shift slightly: our pan band is not matched to the colours
- Use it for display; measure on the original bands



IN pan.tif (1005 × 784) + three coarse colour bands (251 × 200)
OUT sharp.tif · colour at the pan band's detail (1005 × 784)

```
gdal_calc.py -A B04.tif -B B03.tif \
  -C B02.tif --type=Byte \
  --calc="(A.astype(float)+B+C)/3" \
  --outfile=pan.tif

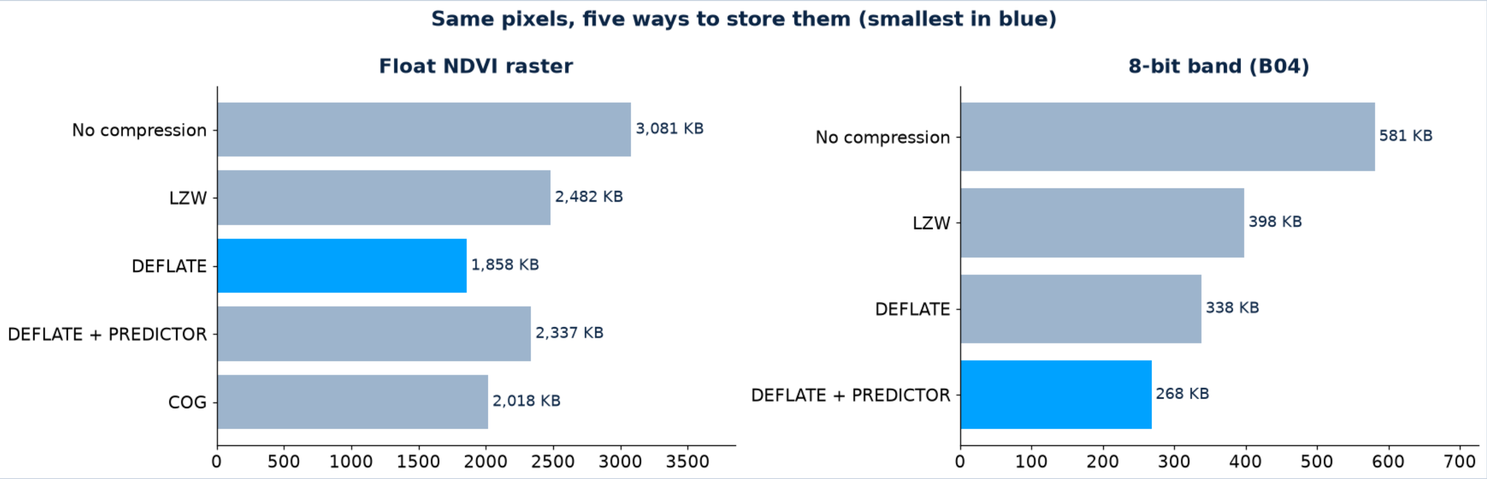
gdal_pansharpen.py -r cubic \
  pan.tif c_B04.tif c_B03.tif \
  c_B02.tif sharp.tif
```

Every number on this slide was measured by running the command on the lab's Sentinel-2 scenes (optical_data) or on one Sentinel-2 scene read straight from the cloud.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Deliver files others can open

COG · compression · NoData · georeferencing



- COG for anything you share: tiles + overviews, readable straight over HTTP
- Float NDVI: DEFLATE 1,858 KB beat DEFLATE+PREDICTOR 2,337 KB
- 8-bit band: PREDICTOR=2 won, 338 KB → 268 KB
- Declare NoData: mean 31.8 (border counted) vs 48.1 (declared)
- Georeference: -a_ullr assigns (no pixel touched) · GCPs + gdalwarp fit a scan (412 × 323 px)

```
                                                    gdal_translate -of COG -co COMPRESS=DEFLATE ndvi.tif ndvi_cog.tif
gdal_translate -co COMPRESS=DEFLATE -co PREDICTOR=2 b04.tif b04_small.tif
gdal_edit.py -a_nodata 0 clip.tif
gdal_translate -a_srs EPSG:4326 -a_ullr 54.35 24.48 54.39 24.45 scan.png scan.tif
gdal_translate -gcp col row x y … scan.png scan_gcp.tif && gdalwarp -order 1 -t_srs EPSG:32640 scan_gcp.tif out.tif
```

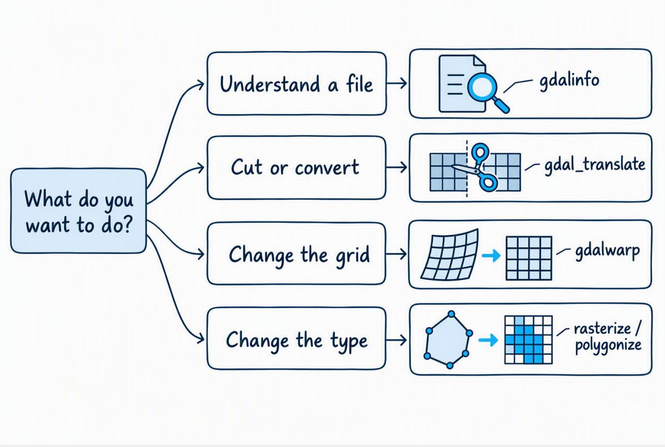
Measure, don't assume: the best compression depends on the data type and the data itself.

This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.

Cheat sheet, and what to do next

Job	Command	Example
Understand a file	gdalinfo -stats -json	1
Cut a window	gdal_translate -srcwin / -projwin	2
Convert format, pick bands	gdal_translate -of -b -scale	3, 5
Stack bands, mosaic free	gdalbuildvrt -separate	4, 10
Index, classes, masks	gdal_calc.py --calc="…" (cast to float)	6, 7
Reproject, resample	gdalwarp -t_srs -tr -r	8
Clip to a boundary	gdalwarp -cutline -crop_to_cutline	9
Read cloud files	/vsicurl/ + COG	11
Overviews, training chips	gdaladdo · gdal_retile.py	12
Vector ↔ raster	gdal_rasterize · gdal_polygonize.py	13, 14
Clean a mask, contour	gdal_sieve.py · gdal_contour	14
Sharpen with a grey band	gdal_pansharpen.py	15
Deliver, fix metadata	gdal_translate -of COG · gdal_edit.py	22

- Next: open GDAL_End_to_End_Lab.ipynb and run all 15 examples yourself
- Three rules to keep: gdalinfo first · cast to float in band maths · ship a COG
- Every command here also works on your own data: change the file name, nothing else



This information is prepared for the National Space Academy, DISTRIBUTION IS PROHIBITED without the written consent of the UAESA.